A) Problem Statement

To classify handwritten digit images (0–9) accurately using both traditional machine learning (Support Vector Machine) and Deep Learning (Convolutional Neural Network) models.

B) EDA and Data Pre-processing Steps Taken

→ The digits dataset from sklearn.datasets was loaded, containing 8x8 pixel images of handwritten digits (0–9). The dataset had no missing values, so we directly moved into exploring and preparing the data.

→ Basic structure and statistics were reviewed using .info() and .describe(), followed by visual checks of class distribution to ensure a balanced dataset. Sample digit images were plotted to confirm label integrity and get a sense of pixel-level patterns.

→ To understand feature relationships, a correlation heatmap was created using seaborn, and pixel intensity patterns were visualized through histograms and boxplots. A pairplot was also used on the first few features to observe how well they separate across classes.

→ The dataset was split into training and test sets (80-20) using stratified sampling to preserve class proportions.

→ For SVM, pixel features were standardized using StandardScaler since SVMs are sensitive to scale. For CNN, image data was normalized (divided by 16.0) and reshaped to (8, 8, 1) to match the expected input shape for convolutional layers.

C) Model Training and Evaluation

→ A train-test split of 80-20 is designated for both the SVM and CNN models. Stratification is applied to ensure all digit classes (0–9) are equally represented in both sets. A fixed random_state is used for reproducibility.

→ For the SVM model, pixel data is first standardized using StandardScaler because SVMs are sensitive to feature scale. An RBF (Radial Basis Function) kernel is used as it handles non-linear decision boundaries well. The model is then trained and evaluated using accuracy score, classification report, and a confusion matrix. ROC curves are plotted for each class to assess performance more granularly.

→ For the CNN model, image data is normalized (values scaled to [0, 1]) and reshaped to include a single grayscale channel (8x8x1). The CNN uses a basic architecture with a convolutional layer (for feature extraction), a max pooling layer (for downsampling), followed by flattening and dense layers. The final output layer uses softmax activation to handle multi-class classification.

→ The model is compiled using the Adam optimizer (adaptive learning rate) and the sparse_categorical_crossentropy loss function (since labels are integers). Model performance is evaluated on the test set, and training history is stored to visualize accuracy and loss trends over the epochs.

→ Accuracy and loss graphs are plotted side-by-side to observe how the model learns and whether it overfits or underfits across the 10 training epochs.

D) Challenges Faced

→ During evaluation of the SVM model, plotting ROC curves required transforming the multiclass labels into a binarized format and extracting decision function scores — this needed extra care to ensure shape alignment across arrays.

→ The CNN model initially did not accept the 8x8 input shape until the channel dimension was added (8x8x1). Proper reshaping was key for TensorFlow to accept the input format.

→ Training history was not stored in the first CNN training call, so the model had to be re-trained with verbose=0 to silently collect the history for plotting purposes.

→ Visualizing and interpreting multi-class ROC curves involved iterating through all digit classes, which was computationally heavier and required fine-tuning of matplotlib plotting settings to keep the output readable.

E) Summary

Machine Learning Model Used: Support Vector Machine (RBF Kernel)

Deep Learning Model Used: Convolutional Neural Network (CNN)

SVM Accuracy: ~ 97.5%

CNN Accuracy: ~ 98% (after 10 epochs)

Key library methods in use →

✓ **Scikit-learn:** SVC, roc_curve, auc, label_binarize

✓ **tensorflow.keras**: Sequential, Conv2D, MaxPooling2D, Flatten, Dense

✓ **matplotlib, seaborn:** Visualization of heatmaps, confusion matrix, training history, and ROC curves

✓ **pandas, numpy:** Data handling and reshaping